

# Tests back : MockMvc, RANDOM\_PORT, RestAssured

Ces trois notions servent à tester une API Spring Boot, mais à des niveaux différents. `MockMvc` et `RestAssured` sont des **clients de test**. `RANDOM_PORT` n'est pas un outil : c'est un **réglage du contexte Spring** qui décide si un vrai serveur démarre.

## MockMvc

Client de test fourni par Spring ( `spring-test` ). Il **simule** des requêtes HTTP sans démarrer de serveur ni ouvrir de port.

- La requête est envoyée directement à la `DispatcherServlet`, en mémoire.
- Le routage, la désérialisation JSON, la validation, la sécurité et les `@ExceptionHandler` passent par Spring. Seule la couche réseau (Tomcat, sockets) est court-circuitée.
- Rapide, et aucune dépendance en plus.

```
mockMvc.perform(post("/api/files").file(multipart))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.name").value("test.txt"));
```

## RANDOM\_PORT

Valeur de `@SpringBootTest(webEnvironment = ...)`. Elle décide comment le contexte Spring démarre pendant le test.

| Mode                       | Comportement                                                                                   |
|----------------------------|------------------------------------------------------------------------------------------------|
| <code>MOCK</code> (défaut) | Environnement web simulé, pas de serveur. C'est le mode à utiliser avec <code>MockMvc</code> . |
| <code>RANDOM_PORT</code>   | Vrai serveur embarqué (Tomcat) sur un port libre choisi au hasard.                             |
| <code>DEFINED_PORT</code>  | Vrai serveur, sur le port de la configuration (par exemple 8080).                              |
| <code>NONE</code>          | Pas d'environnement web du tout.                                                               |

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class MyIT { @LocalServerPort int port; ... }
```

Le port est aléatoire pour éviter les conflits quand plusieurs tests ou builds tournent en parallèle. `@LocalServerPort` injecte le port réellement choisi.

## RestAssured

Bibliothèque Java externe, à la syntaxe fluide `given / when / then`. Elle envoie de **vraies requêtes HTTP** vers une URL. Il lui faut donc un serveur qui tourne, d'où son association habituelle avec `RANDOM_PORT`.

```
given().port(port)
    .multiPart("file", file)
.when().post("/api/files")
.then().statusCode(201)
    .body("name", equalTo("test.txt"));
```

## Comparaison

|                                                          | MockMvc       | RestAssured + RANDOM_PORT |
|----------------------------------------------------------|---------------|---------------------------|
| Serveur réel                                             | Non           | Oui (Tomcat)              |
| Passe par le réseau                                      | Non           | Oui (localhost)           |
| Vitesse                                                  | Rapide        | Plus lent                 |
| Filtres servlet, config Tomcat, limites d'upload réelles | Partiellement | Oui                       |
| Dépendance à ajouter                                     | Non           | Oui                       |

## Dans DataShare

- Les tests d'intégration du back utilisent **MockMvc en mode MOCK** (`AbstractIntegrationTest`, `FileControllerIT`).
- `RestAssured` n'est pas déclaré dans `backend/pom.xml` et aucun test n'utilise `RANDOM_PORT`.
- Un audit a relevé que `README.md` et `TESTING.md` annonçaient l'inverse. La doc a été corrigée pour décrire MockMvc.

## Quand ajouter RestAssured + RANDOM\_PORT

MockMvc suffit tant qu'on teste la logique Spring. Un vrai serveur devient utile pour ce que MockMvc ne voit pas :

- la taille maximale d'upload appliquée par Tomcat ;
- les filtres servlet et la configuration du conteneur ;
- le comportement réel de bout en bout (en-têtes, encodage, connexions).

## À retenir

1. MockMvc simule HTTP, sans réseau : rapide, il couvre l'essentiel de Spring.
2. `RANDOM_PORT` démarre un vrai serveur sur un port libre.
3. RestAssured a besoin d'un serveur, donc de `RANDOM_PORT`.
4. Le mode par défaut de `@SpringBootTest` est `MOCK`, celui de MockMvc.